

Hybrid Agent Report

class Ontology:

This class has a subclass called Predicate that has a list containing possible propositions. The *split_proposition* method parses the input and *validate* checks if it's valid. And *NOT* gives the inverse of a proposition.

class KnowledgeBase:

This class has the subclass Rule.

The initializer of KnowledgeBase initializes blocking rules and granting rules with the method *initialize_default_rules* and has *add_fact*.

class FOLEngine:

All the methods in this class are used for Forward Chaining and Unification algorithms. The method *FOL_FC_Ask* adds the ground facts to the knowledge base.

class PropositionalEngine:

The methods are used for Resolution algorithm. The *final_decision* method checks if any permission is granted and no blocking rule exists using Resolution algorithm.

Bonus:

1. Contradiction handling in *add_fact*
2. Working hour: the blocking rule "NOT Time(start, end) → NOT Access(room)" is added

Examples

Input:

Facts:

Student("Alice")

HasID("Alice")

Professor("DrSmith")

Mentor("DrSmith", "Alice")

InRoom("DrSmith", "Lab2")

IsSecureLab("Lab2")

DoorOpen("Lab2")

Time(8)

CanAccess("Alice", "Lab2")

Output:

Phase 1: First-Order Logic Inference (FOL Grounding)

New proposition generated: Student["Alice"] and Mentor["DrSmith", "Alice"] and InRoom["DrSmith", "Lab2"] --> PossibleAccessViaSupervision["Alice", "Lab2"]

Phase 2: Checking Constraints (Propositional Logic)

The door is open

The room is not emergency

It isn't working hour

Conclusion: Blocking rule found, and at least one permission is active.

Final Result: False

Input:

Facts:

Professor("DrSmith")

Dean("DrSmith")

DoorOpen("Lab2")

Time(10)

Output:

Phase 1: First-Order Logic Inference (FOL Grounding)

New proposition generated: Dean[""DrSmith""] --> PossibleAccessViaDean[""DrSmith"", "Lab2"]

Phase 2: Checking Constraints (Propositional Logic)

The door is open

The room is not emergency

It is working hour

Conclusion: No blocking rule found, and at least one permission is active.

Final Result: True

Input:

Facts:

Professor("DrSmith")

Office("DrSmith", "Lab")

Emergency("Lab")

Output:

Phase 1: First-Order Logic Inference (FOL Grounding)

*New proposition generated: Professor["DrSmith"] and Office["DrSmith", "Lab"] -->
PossibleAccessViaOffice["DrSmith", "Lab"]*

Phase 2: Checking Constraints (Propositional Logic)

The door is not open

The room is emergency

Conclusion: Blocking rule found, and at least one permission is active.

Final Result: False

Input:

Facts:

Student("Alice")

MemberOf("Alice", "Project")

ProjectRoom("Project", "Lab")

DoorOpen("Lab")

Output:

Phase 1: First-Order Logic Inference (FOL Grounding)

Phase 2: Checking Constraints (Propositional Logic)

The door is open

The room is not emergency

Conclusion: Blocking rule found, and no permission is active.

Final Result: False

Input:

Facts:

Student("Alice")

MemberOf("Alice", "Project")

ProjectRoom("Project", "Lab")

HasID("Alice")

DoorOpen("Lab")

Output:

Phase 1: First-Order Logic Inference (FOL Grounding)

New proposition generated: Student["Alice"] and MemberOf["Alice", "Project"] and ProjectRoom["Project", "Lab"] and HasID["Alice"] --> PossibleAccessViaProject["Alice", "Lab"]

Phase 2: Checking Constraints (Propositional Logic)

The door is open

The room is not emergency

Conclusion: Blocking rule found, and at least one permission is active.

Final Result: True

Input:

Facts:

Professor("Alice")

Dean("Alice")

IsSecureLab("Lab")

PassedSafetyCourse("Alice")

HasID("Alice")

DoorOpen("Lab")

Time(12)

Output:

Phase 1: First-Order Logic Inference (FOL Grounding)

New proposition generated: Dean["Alice"] --> PossibleAccessViaDean["Alice", "Lab"]

New proposition generated: HasID["Alice"] and IsSecureLab["Lab"] and PassedSafetyCourse["Alice"] --> PossibleAccessViaSecureLab["Alice", "Lab"]

Phase 2: Checking Constraints (Propositional Logic)

The door is open

The room is not emergency

It is working hour

Conclusion: No blocking rule found, and at least one permission is active.

Final Result: True

Input:

Facts:

Professor("Alice")

Student("Alice")

Output:

ValueError: This fact causes contradiction

Input:

Facts:

HasCar("Alice")

Output:

ValueError: Predicate HasCar not defined

Input:

Facts:

Professor("Alice")

InRoom("Lab")

Output:

ValueError: Predicate InRoom expects 2 arguments, got 1